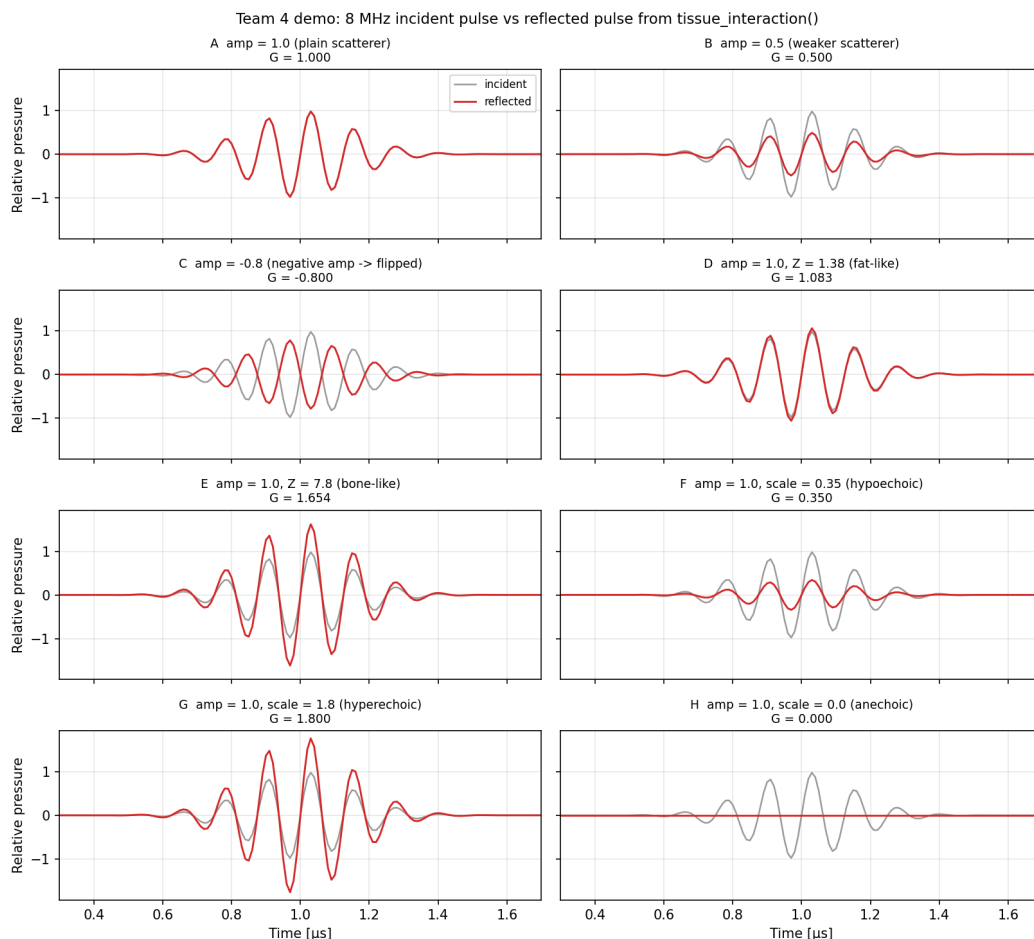


Team 4: Tissue Interaction

How tissue scatters the ultrasound pulse back: impedance, point scatterers and lesions

NITK-USoundSim · 2-D ultrasound simulator · explanation, input, output and run guide

- Team 4 turns the **incident** pressure pulse arriving at a scatterer into the **reflected** (scattered) pulse.
- It also builds a 2-D **liver phantom**: 300 random point scatterers with an optional lesion.
- One command: `python3 run_team4.py`. It runs the team's validation script, the 13 unit tests (13/13 pass), a pulse demo and a phantom demo.
- Runtime about 2.6 s (plus Python start-up). All results go to the output/ folder.



Incident 8 MHz pulse (grey) and reflected pulse (red) for 8 scatterer settings (from the demo run).

Contents

1 Overview	3
Files in the folder	3
2 Theory	4
2.1 Acoustic impedance	4
2.2 Pressure reflection coefficient and its sign	4
2.3 The gain used in the team's code	4
2.4 Point-scatterer tissue model (Field II style)	4
2.5 Scatterer density	5
2.6 Lesions and echo classes	5
2.7 Speckle	5
3 Code explanation	6
3.1 code/tissue_interaction.py	6
3.2 code/phantom.py	6
3.3 code/validate_phantom.py	6
3.4 code/tests/	7
3.5 How run_team4.py uses the team's own code	7
3.6 run_team4.py step by step (written for this demo)	8
3.7 Full code listings	9
4 Input	22
4.1 Incident pulse: input/incident_pulse_8MHz.csv	22
4.2 Scatterer cases: input/demo_scatterers.json	22
4.3 Phantom configurations: input/phantom_configs.json	22
5 Output	24
5.1 Team's validation plots (output/validation_plots/)	24
5.2 Pulse demo (Step 3)	25
5.3 Phantoms (Step 4)	27
5.4 Speckle illustration (Step 5)	28
5.5 Other output files	28
5.6 Console output (real, captured)	28
6 How to run	31
Optional: run the team's files yourself	31
Troubleshooting	31
7 Check yourself	32

1 Overview

What Team 4 does and what is in the folder

An ultrasound probe sends a short pressure pulse into the body. When the pulse meets tissue, a small part of it is sent back (scattered) towards the probe. These echoes are what the scanner turns into an image. **Team 4 models this tissue step.**

- **tissue_interaction():** takes one incident waveform (a 1-D array) and one scatterer (a dictionary with amp and optional impedance / scale). It returns the reflected waveform with the **same shape**, multiplied by a gain G. No time delay is added.
- **pressure_reflection_coefficient():** $R = (Z_2 - Z_1) / (Z_2 + Z_1)$ for two acoustic impedances.
- **generate_liver_phantom():** makes a reproducible 2-D set of point scatterers (random positions, random Gaussian amplitudes) with an optional elliptical lesion.

Place in the pipeline

Transducer and TX beamforming make the pulse, Team 3 (propagation) carries it to each scatterer, **Team 4 scales it into an echo**, Team 3 carries the echo back to the receive elements, and later teams beamform it and form the B-mode image.

Files in the folder

Path	What it is
run_team4.py	One command that runs everything (added for the demo)
requirements.txt	numpy, scipy, matplotlib; pytest (only to run code/tests yourself) (added)
README.md	Step-by-step run instructions (added)
.vscode/settings.json	VS Code setting: adds code/ to the import path so the team's files run with the Run button (added; hidden folder)
code/tissue_interaction.py	Team's original: reflection coefficient, gain, tissue_interaction()
code/phantom.py	Team's original: LesionProperties, PhantomConfig, generate_liver_phantom()
code/validate_phantom.py	Team's original: makes 4 validation plots
code/tests/test_tissue_interaction.py	Team's original: 13 unit tests
code/tests/conftest.py	Team's original: adds code/ to the Python path for the tests
code/README.md	Team's original technical README
input/	incident_pulse_8MHz.csv, demo_scatterers.json, phantom_configs.json (written by the run)
output/	All plots, phantom CSV files, console_log.txt, console_screenshot.png
Team_4_Tissue_Interaction_Explanation.pdf	This document

The files in code/ are byte-for-byte copies of the team's files. The runner only calls them; nothing in them was changed.

2 Theory

The physics behind the code, in simple words

2.1 Acoustic impedance

Every material has an **acoustic impedance** Z . It tells how hard it is to push the material with sound. It is density times speed of sound:

$$Z = \rho c$$

ρ = density [kg/m^3], c = speed of sound [m/s]. Unit: Rayl; 1 MRayl = $10^6 \text{ kg m}^{-2} \text{ s}^{-1}$.

Soft tissue (like liver) is about 1.63 MRayl; this is the background value used by the team's code. Where two materials with different Z meet, part of the wave is reflected.

2.2 Pressure reflection coefficient and its sign

$$R = \frac{Z_2 - Z_1}{Z_2 + Z_1}$$

Normal incidence: wave travels in material 1 and hits material 2.

- $R = 0$ when $Z_2 = Z_1$: no reflection (same tissue).
- $R > 0$ when $Z_2 > Z_1$ (e.g. tissue to bone): the echo has the **same sign** as the incident pulse.
- $R < 0$ when $Z_2 < Z_1$ (e.g. tissue to fat or air): the echo is **phase inverted** (flipped upside down).
- $|R|$ is between 0 and 1. The fraction of energy reflected is R^2 . Tissue-to-air gives $|R|$ close to 1, which is why ultrasound gel is needed.

2.3 The gain used in the team's code

$$G = \text{amp} \times (1 + |R|) \times \text{interaction_scale}$$

$$p_{\text{reflected}}(t) = G p_{\text{incident}}(t)$$

- **amp** is the main, signed strength of the point scatterer. A negative amp flips the echo.
- **(1 + |R|)** is used only if the scatterer carries its own impedance. It gives a *modest* boost (1.0 to 2.0). The team chose this so every tiny scatterer is not treated like a big flat boundary.
- **interaction_scale** (default 1) is an extra non-negative multiplier for calibration.
- The model is linear: the output is just the input times one number. It does not change the pulse shape and does not add any delay; delays and attenuation belong to the propagation team.

2.4 Point-scatterer tissue model (Field II style)

Real tissue has cells, fibres and small structures much smaller than the wavelength ($\lambda = c/f \approx 0.19 \text{ mm}$ at 8 MHz). We cannot model each one. Instead, like the well-known Field II simulator, tissue is represented by many **point scatterers**. Each has a position (x, z) and an amplitude. The received signal is the sum of the echoes from all scatterers.

- **Positions are uniform random**: tissue has no preferred place for scatterers, so every point in the region is equally likely.
- **Amplitudes are Gaussian random, mean 0**: the small structures are slightly stronger or weaker than their surroundings, in both directions, so the sign is random. Many small independent effects add to a Gaussian (central limit theorem).

- **Seed:** a fixed random seed (2026) makes the phantom exactly the same every run.

2.5 Scatterer density

$$N = \text{round}(\text{density} \times \text{area})$$

Default: $120,000 \text{ per m}^2 \times (50 \text{ mm} \times 50 \text{ mm} = 0.0025 \text{ m}^2) = 300 \text{ scatterers}$.

More scatterers per resolution cell give more realistic, fully developed speckle (textbooks often say about 10 or more per resolution cell). The team's default of 300 in 25 cm^2 is a light-weight simulation setting (their README says it is not a histological model of liver).

2.6 Lesions and echo classes

A lesion is an ellipse: a scatterer is inside if $((x-cx)/rx)^2 + ((z-cz)/rz)^2 \leq 1$. The amplitudes of scatterers inside are multiplied by a scale factor:

Echo class	Meaning on the image	Scale in phantom.py
hypoechoic	darker than surrounding tissue (e.g. some tumours)	0.35
isoechoic	same brightness as surrounding tissue	1.0
hyperechoic	brighter than surrounding tissue (e.g. haemangioma)	1.8
anechoic	no echoes, black (e.g. a fluid-filled cyst)	scattering_scale = 0.0

The lesion lives in the **acoustic phantom**, not in the image: no image pixels are edited.

2.7 Speckle

Many scatterers lie inside one pulse length. Their echoes arrive at nearly the same time with random amplitudes and signs, so they add up sometimes constructively (bright) and sometimes destructively (dark). This random granular pattern is called **speckle**. It is not noise from electronics; it comes from interference of sub-wavelength scatterers, and it is why a random scatterer phantom looks like real tissue.

3 Code explanation

Every file, section by section, then the full listings

3.1 code/tissue_interaction.py

- **Module docstring:** says this is a drop-in replacement for the old placeholder `mock_tissue_interaction(incident_wave, scatterer)`; same input and output shape; no delays, attenuation or beamforming here.
- **TissueProperties** (dataclass): stores default values: impedance 1.63 MRayl, background 1.63 MRayl, amplitude 1.0. It documents units; the other functions do not use it directly.
- **pressure_reflection_coefficient(z1, z2):** checks both impedances are finite and positive (otherwise `ValueError`), then returns $(z2 - z1)/(z2 + z1)$ with its sign.
- **_validate_amplitude(amp):** turns amp into a float and rejects NaN / infinity.
- **interaction_gain(scatterer):** reads amp (default 1.0). If `impedance_mrayl` is given, it also needs `background_impedance_mrayl`; it computes $R(\text{background} \rightarrow \text{local})$ and a factor $1 + |R|$. Then it reads `interaction_scale` (default 1.0; must be finite and ≥ 0) and returns $\text{amp} \times \text{factor} \times \text{scale}$.
- **tissue_interaction(incident_wave, scatterer):** converts the input to a float array, checks it is 1-D and finite, multiplies by the gain, checks the result is finite, and returns it (same shape).
- **mock_tissue_interaction = tissue_interaction:** keeps the old name so the propagation code still works.

3.2 code/phantom.py

- **LesionProperties:** centre (default $x = 0$, $z = 30$ mm), radii (6 mm), echo class, optional `scattering_scale` override and optional impedance. `__post_init__` rejects bad values (non-positive radii, unknown class, negative scale or impedance).
- **PhantomConfig:** region $x = -25..25$ mm, $z = 10..60$ mm, density 120,000 per m^2 , background impedance 1.63 MRayl, amplitude std 1.0, seed 2026, optional lesion. It also validates its values.
- **Phantom2D:** the result: region limits, seed, the list of scatterer dictionaries and the lesion. `num_scatterers` gives the count; `as_legacy_scatterers()` returns only x , z , amp for older propagation code.
- **_inside_ellipse:** True for points inside the ellipse equation.
- **_lesion_scale:** returns the override if given, else 0.35 / 1.0 / 1.8 for hypo / iso / hyper.
- **generate_liver_phantom(config):** makes a random generator with the seed, computes $N = \text{round}(\text{density} \times \text{area})$, draws x and z uniformly and amp from $\text{Normal}(0, \text{std})$. Scatterers inside the lesion get $\text{amp} \times \text{scale}$. Each scatterer becomes a dictionary with x , z , amp, background impedance, `is_lesion` and (if set) the lesion impedance.

3.3 code/validate_phantom.py

- Builds a hypoechoic lesion (centre $z = 35$ mm, radii 7×8 mm) and a phantom with seed 2026 and density 120,000.
- Plot 1: scatterer positions, lesion scatterers highlighted, lesion outline.
- Plot 2: histogram of amplitudes, background vs lesion.
- Plot 3: a 5 MHz Gaussian pulse ($f_s = 40$ MHz, 256 samples) passed through `tissue_interaction()` using the middle scatterer of the list.

- Plot 4: map of $|\text{amp}|$ over the region.
- It saves the PNGs into a folder `validation_plots/` in the current working folder and prints the scatterer counts. The runner starts it with `output/` as working folder.

3.4 code/tests/

`confest.py` only adds `code/` to the Python path. `test_tissue_interaction.py` has 13 tests: $R = 0$ for equal impedance; R matches the formula; amp-only scatterer gives $0.4 \times \text{wave}$; shape preserved; impedance contrast raises the gain; empty phantom has 0 scatterers; single scatterer echo; two scatterers add linearly; same seed gives the same phantom; different seed gives a different one; hypoechoic lesion has smaller amplitude spread; scatterers stay inside the region and are finite; a 2-D waveform is rejected.

3.5 How `run_team4.py` uses the team's own code

In one sentence

The algorithm and all the Team 4 logic (reflection coefficient, gain, `tissue_interaction()`, phantom generation, validation script and unit tests) come from the team's existing code in `code/`, which is used **unchanged** (it was not rewritten). `run_team4.py` is only a **demo wrapper**: it imports or starts the team's files, feeds them sample inputs and saves the results.

Where `run_team4.py` uses your code

run_team4.py line(s)	Team file / function used	What it does there
39, 44	folder <code>code/</code>	Points to <code>code/</code> and puts it on the Python import path, so the imports below load the team's original files.
46	<code>phantom.py</code> : <code>LesionProperties</code> , <code>PhantomConfig</code> , <code>generate_liver_phantom</code>	Imports the team's phantom code.
47-49	<code>tissue_interaction.py</code> : <code>interaction_gain</code> , <code>pressure_reflection_coefficient</code> , <code>tissue_interaction</code>	Imports the team's tissue model.
108-109	<code>validate_phantom.py</code>	Step 1: runs the team's validation script as it is (separate Python process, working folder <code>output/</code>).
116	<code>validate_phantom.py</code> (its plots)	Collects the PNG files the script wrote into <code>output/validation_plots/</code> .
152	<code>tests/confest.py</code>	Step 2: loads the team's <code>confest</code> (adds <code>code/</code> to the path).
153	<code>tests/test_tissue_interaction.py</code>	Step 2: loads the team's test file.
159	the 13 <code>test_</code> functions in that file	Calls each of the team's tests, one by one.
204	<code>tissue_interaction()</code>	Step 3: makes the reflected pulse for each of the 8 scatterer cases.
205, 215	<code>interaction_gain()</code>	Step 3: gets the gain G for the printed table and the plot titles.
241	<code>pressure_reflection_coefficient()</code>	Step 3: R from soft tissue (1.63 MRayl) into each example material.
263-266	<code>LesionProperties</code>	Step 4: builds the hypo-, iso-, hyperechoic and anechoic lesions.
272	<code>PhantomConfig</code>	Step 4: builds the phantom settings (seed, density, lesion).

run_team4.py line(s)	Team file / function used	What it does there
273	generate_liver_phantom()	Step 4: makes each of the 5 phantoms.
354	tissue_interaction()	Step 5: makes the echo of each of the 80 scatterers in the speckle demo.

What run_team4.py adds on top (written for this demo, not the team's code)

run_team4.py line(s)	Demo addition
53-69, 79-94, 387-389	Console log: everything printed is also written to output/console_log.txt; at the end it is drawn as output/console_screenshot.png.
123-142	Running the tests without pytest: if pytest is missing, a small stand-in for pytest.raises is made (the only pytest feature the tests use).
145-149, 154-165	Own test loop: loads the test files, finds the test_ functions, prints PASS/FAIL and counts them.
169-183	Step 3 incident pulse: the runner's own calculation of an 8 MHz Gaussian-modulated sine ($f_s = 100$ MHz, 200 samples, $\sigma = 0.15$ μ s), saved to input/incident_pulse_8MHz.csv.
185-199	The 8 scatterer cases A-H (example values chosen for the demo), saved to input/demo_scatterers.json.
236-237, 243-244	Example impedances for fat, water, blood, muscle, bone, air (textbook-style values chosen for the demo); the runner itself computes R^2 , $1 + R $ and the sign label for the printout.
260-268, 276-278	Phantom settings chosen for the demo (lesion at (0, 35) mm, radii 7×8 mm, density 120,000, seed 2026), saved to input/phantom_configs.json.
280-296	Own checks printed in Step 4: region area, expected $N = \text{round}(\text{density} \times \text{area})$, a copy of the scale values $0.35 / 1.0 / 1.8$ (line 287) for the table, and std(background), std(lesion) and their ratio.
297-301	CSV dumps: output/phantom_<type>_scatterers.csv.
344-357	Step 5 speckle: 80 random scatterers (seed 7, depths 1.5-7.5 mm); the runner adds the round-trip delay $2z/c$ and sums the echoes. Delays are not part of Team 4's code (propagation is Team 3's job).
358-363	Hilbert envelope (scipy.signal.hilbert) for the plot, the wavelength, scatterers per wavelength and the envelope mean and std/mean.
210-231, 246-256, 305-338, 364-373	Extra figures demo_00 to demo_05 (matplotlib).
376-385	Summary: list of output files and total runtime.

To see it yourself: open run_team4.py at the lines above; Cmd/Ctrl+Click a team function in VS Code opens the original file in code/. The line numbers are the real line numbers of the file, as in the listing in 3.7.

3.6 run_team4.py step by step (written for this demo)

- Copies everything printed to output/console_log.txt and at the end draws it as console_screenshot.png.
- Step 1: runs validate_phantom.py as a separate process inside output/.
- Step 2: loads conftest.py and the test file and calls each test_ function, printing PASS/FAIL. pytest is not needed: if it is missing, a tiny stand-in for pytest.raises (the only pytest feature the tests use) is provided.

- Step 3: builds the 8 MHz input pulse, saves it, runs `tissue_interaction()` for 8 scatterer cases, prints gains, and prints R for example impedances.
- Step 4: generates 5 phantoms with `generate_liver_phantom()` and plots them.
- Step 5: a 1-D speckle illustration. The runner (not Team 4's code) adds the round-trip delays $2z/c$ here, only to show interference; in the simulator delays are the propagation team's job.

3.7 Full code listings

code/tissue_interaction.py

```

1  """Tissue interaction model for NITK-UsoundSim.
2
3  This module is intentionally a drop-in replacement for the existing
4  Acoustic Propagation placeholder::
5
6      reflected_wave = mock_tissue_interaction(incident_wave, scatterer)
7
8  The propagation contract is preserved: one scatterer dictionary is supplied,
9  and a 1-D scattered pressure waveform with the same shape is returned.
10
11  The model is linear and point-scatterer based. It does not calculate TX/RX
12  travel time, attenuation, beamforming, RF demodulation, or B-mode formation.
13  Those responsibilities remain with the existing propagation/receive pipeline.
14  """
15
16  from __future__ import annotations
17
18  from dataclasses import dataclass
19  from typing import Mapping
20
21  import numpy as np
22
23
24  @dataclass(frozen=True)
25  class TissueProperties:
26      """Acoustic properties used by the interaction model.
27
28      Units
29      -----
30      impedance_mrayl : MRayl (10^6 kg m^-2 s^-1)
31      background_impedance_mrayl : MRayl
32      scatterer_amplitude : dimensionless signed local scattering strength
33      """
34
35      impedance_mrayl: float = 1.63
36      background_impedance_mrayl: float = 1.63
37      scatterer_amplitude: float = 1.0
38
39
40  def pressure_reflection_coefficient(z1_mrayl: float, z2_mrayl: float) -> float:
41      """Return the normal-incidence pressure reflection coefficient.
42
43      
$$R = (Z_2 - Z_1) / (Z_2 + Z_1)$$

44
45      The sign is retained because a negative pressure reflection coefficient
46      represents phase inversion for a lower-impedance transition.
47      """
48      z1 = float(z1_mrayl)
49      z2 = float(z2_mrayl)
50      if not np.isfinite(z1) or not np.isfinite(z2) or z1 <= 0.0 or z2 <= 0.0:
51          raise ValueError("Acoustic impedances must be finite and positive.")
52      return float((z2 - z1) / (z2 + z1))
53
54
55  def _validate_amplitude(amp: float) -> float:
56      value = float(amp)
57      if not np.isfinite(value):
58          raise ValueError("scatterer['amp'] must be finite.")
59      return value
60
61
62  def interaction_gain(scatterer: Mapping[str, float]) -> float:
63      """Return the dimensionless gain applied to the incident waveform.
64

```

```

65     Required legacy field
66     -----
67     ``amp`` is the local point-scatterer strength. If it is omitted, 1.0 is
68     used, matching the preliminary module.
69
70     Optional fields
71     -----
72     ``impedance_mrayl`` and ``background_impedance_mrayl`` can be supplied to
73     modestly modulate the magnitude using ``1 + |R|``. This preserves the
74     original test/interface behaviour and avoids treating every distributed
75     scatterer as a macroscopic tissue boundary.
76
77     ``interaction_scale`` is an optional dimensionless multiplier useful for
78     lesion/background calibration. It changes the acoustic scatterer response,
79     not the final image brightness.
80     """
81     amp = _validate_amplitude(scatterer.get("amp", 1.0))
82
83     z_local = scatterer.get("impedance_mrayl")
84     z_bg = scatterer.get("background_impedance_mrayl")
85     impedance_factor = 1.0
86     if z_local is not None:
87         if z_bg is None:
88             raise ValueError(
89                 "background_impedance_mrayl is required when impedance_mrayl is supplied."
90             )
91         r = pressure_reflection_coefficient(float(z_bg), float(z_local))
92         impedance_factor = 1.0 + abs(r)
93
94     scale = float(scatterer.get("interaction_scale", 1.0))
95     if not np.isfinite(scale) or scale < 0.0:
96         raise ValueError("scatterer['interaction_scale'] must be finite and non-negative.")
97
98     return amp * impedance_factor * scale
99
100
101 def tissue_interaction(
102     incident_wave: np.ndarray,
103     scatterer: Mapping[str, float],
104 ) -> np.ndarray:
105     """Convert one incident pressure waveform into a scattered waveform.
106
107     Interface contract
108     -----
109     input  : 1-D mechanical-pressure waveform, shape ``(n_samples,)``
110     output : 1-D mechanical-pressure waveform, exactly the same shape
111
112     No time shift, TX delay, RX delay, attenuation, beamforming, or image
113     processing is applied here. The existing return-propagation module should
114     continue to handle the scatterer-to-receiver path.
115     """
116     wave = np.asarray(incident_wave, dtype=float)
117     if wave.ndim != 1:
118         raise ValueError("incident_wave must be a 1-D waveform.")
119     if not np.all(np.isfinite(wave)):
120         raise ValueError("incident_wave must contain only finite values.")
121
122     gain = interaction_gain(scatterer)
123     out = gain * wave
124     if not np.all(np.isfinite(out)):
125         raise FloatingPointError("Tissue interaction produced non-finite values.")
126     return out
127
128
129 # Exact legacy name expected by the current propagation code.
130 mock_tissue_interaction = tissue_interaction
131
132
133 __all__ = [
134     "TissueProperties",
135     "pressure_reflection_coefficient",
136     "interaction_gain",
137     "tissue_interaction",
138     "mock_tissue_interaction",
139 ]

```

code/phantom.py

```

1  """Deterministic 2-D liver-like acoustic phantom generation for NITK-UsoundSim.
2
3  The phantom is an acoustic-domain representation. It contains distributed
4  point scatterers and an optional geometrical lesion. It does not create or
5  modify a B-mode image.
6  """
7
8  from __future__ import annotations
9
10 from dataclasses import dataclass
11 from typing import Literal
12
13 import numpy as np
14
15
16 LesionEchoClass = Literal["hypoechoic", "isoechoic", "hyperechoic"]
17
18
19 @dataclass(frozen=True)
20 class LesionProperties:
21     """Geometrical and acoustic/scattering properties of a focal lesion."""
22
23     center_x_m: float = 0.0
24     center_z_m: float = 0.030
25     radius_x_m: float = 0.006
26     radius_z_m: float = 0.006
27     echo_class: LesionEchoClass = "hypoechoic"
28     scattering_scale: float | None = None
29     impedance_mrayl: float | None = None
30
31     def __post_init__(self) -> None:
32         if self.radius_x_m <= 0 or self.radius_z_m <= 0:
33             raise ValueError("Lesion radii must be positive.")
34         if self.echo_class not in {"hypoechoic", "isoechoic", "hyperechoic"}:
35             raise ValueError("echo_class must be hypoechoic, isoechoic, or hyperechoic.")
36         if self.scattering_scale is not None and self.scattering_scale < 0:
37             raise ValueError("scattering_scale must be non-negative.")
38         if self.impedance_mrayl is not None and self.impedance_mrayl <= 0:
39             raise ValueError("impedance_mrayl must be positive.")
40
41
42 @dataclass(frozen=True)
43 class PhantomConfig:
44     """Configuration for the 2-D synthetic liver phantom.
45
46     All spatial quantities are SI metres. ``scatterer_density_m2`` is an
47     intentionally synthetic simulation parameter, not a claim about human
48     liver microstructure.
49     """
50
51     x_min_m: float = -0.025
52     x_max_m: float = 0.025
53     z_min_m: float = 0.010
54     z_max_m: float = 0.060
55     scatterer_density_m2: float = 120_000.0
56     background_impedance_mrayl: float = 1.63
57     background_scatter_std: float = 1.0
58     seed: int = 2026
59     lesion: LesionProperties | None = None
60
61     def __post_init__(self) -> None:
62         if self.x_max_m <= self.x_min_m or self.z_max_m <= self.z_min_m:
63             raise ValueError("Phantom bounds must have positive extent.")
64         if self.scatterer_density_m2 < 0:
65             raise ValueError("scatterer_density_m2 must be non-negative.")
66         if self.background_impedance_mrayl <= 0:
67             raise ValueError("background_impedance_mrayl must be positive.")
68         if self.background_scatter_std < 0:
69             raise ValueError("background_scatter_std must be non-negative.")
70
71
72 @dataclass
73 class Phantom2D:
74     """Generated acoustic phantom and propagation-facing scatterer list."""
75
76     x_min_m: float

```

```

77     x_max_m: float
78     z_min_m: float
79     z_max_m: float
80     background_impedance_mrayl: float
81     seed: int
82     scatterers: list[dict[str, float | str | bool]]
83     lesion: LesionProperties | None
84
85     @property
86     def num_scatterers(self) -> int:
87         return len(self.scatterers)
88
89     def as_legacy_scatterers(self) -> list[dict[str, float]]:
90         """Return only the fields needed by the existing propagation code.
91
92         The current propagation README says a scatterer contains at least
93         ``x``, ``z`` and ``amp``. Extra metadata may safely be retained in the
94         richer list, but this helper provides a clean legacy view.
95         """
96         return [
97             {"x": float(s["x"]), "z": float(s["z"]), "amp": float(s["amp"])}
98             for s in self.scatterers
99         ]
100
101
102 def _inside_ellipse(x: np.ndarray, z: np.ndarray, lesion: LesionProperties) -> np.ndarray:
103     qx = (x - lesion.center_x_m) / lesion.radius_x_m
104     qz = (z - lesion.center_z_m) / lesion.radius_z_m
105     return qx * qx + qz * qz <= 1.0
106
107
108 def _lesion_scale(lesion: LesionProperties, background_std: float) -> float:
109     if lesion.scattering_scale is not None:
110         return float(lesion.scattering_scale)
111     if lesion.echo_class == "hypoechoic":
112         return 0.35
113     if lesion.echo_class == "isoechoic":
114         return 1.0
115     return 1.8
116
117
118 def generate_liver_phantom(config: PhantomConfig = PhantomConfig()) -> Phantom2D:
119     """Generate a reproducible liver-like distributed point-scatterer phantom.
120
121     Scatterer positions are uniform in the rectangular imaging region.
122     Signed scatterer amplitudes are sampled from a zero-mean Gaussian. The
123     lesion changes the local scattering-strength scale and can optionally
124     carry a local impedance for the tissue interaction adapter.
125     """
126     rng = np.random.default_rng(config.seed)
127     area_m2 = (config.x_max_m - config.x_min_m) * (config.z_max_m - config.z_min_m)
128     n = int(np.round(config.scatterer_density_m2 * area_m2))
129
130     x = rng.uniform(config.x_min_m, config.x_max_m, n)
131     z = rng.uniform(config.z_min_m, config.z_max_m, n)
132     amp = rng.normal(0.0, config.background_scatter_std, n)
133
134     inside = np.zeros(n, dtype=bool)
135     if config.lesion is not None and n:
136         inside = _inside_ellipse(x, z, config.lesion)
137         amp[inside] *= _lesion_scale(config.lesion, config.background_scatter_std)
138
139     scatterers: list[dict[str, float | str | bool]] = []
140     for i in range(n):
141         item: dict[str, float | str | bool] = {
142             "x": float(x[i]),
143             "z": float(z[i]),
144             "amp": float(amp[i]),
145             "background_impedance_mrayl": float(config.background_impedance_mrayl),
146             "is_lesion": bool(inside[i]),
147         }
148         if inside[i] and config.lesion is not None and config.lesion.impedance_mrayl is not None:
149             item["impedance_mrayl"] = float(config.lesion.impedance_mrayl)
150         scatterers.append(item)
151
152     return Phantom2D(
153         x_min_m=config.x_min_m,

```

```

154         x_max_m=config.x_max_m,
155         z_min_m=config.z_min_m,
156         z_max_m=config.z_max_m,
157         background_impedance_mrayl=config.background_impedance_mrayl,
158         seed=config.seed,
159         scatterers=scatterers,
160         lesion=config.lesion,
161     )
162
163
164 __all__ = ["LesionProperties", "PhantomConfig", "Phantom2D", "generate_liver_phantom"]

```

code/validate_phantom.py

```

1  """Create diagnostic validation plots for the Tissue Interaction module."""
2  from pathlib import Path
3  import numpy as np
4  import matplotlib.pyplot as plt
5
6  from phantom import LesionProperties, PhantomConfig, generate_liver_phantom
7  from tissue_interaction import tissue_interaction
8
9  OUT = Path("validation_plots")
10 OUT.mkdir(exist_ok=True)
11
12 lesion = LesionProperties(
13     center_x_m=0.0,
14     center_z_m=0.035,
15     radius_x_m=0.007,
16     radius_z_m=0.008,
17     echo_class="hypoechoic",
18 )
19 cfg = PhantomConfig(seed=2026, scatterer_density_m2=120_000, lesion=lesion)
20 phantom = generate_liver_phantom(cfg)
21
22 x = np.array([s["x"] for s in phantom.scatterers], dtype=float)
23 z = np.array([s["z"] for s in phantom.scatterers], dtype=float)
24 a = np.array([s["amp"] for s in phantom.scatterers], dtype=float)
25 is_lesion = np.array([s["is_lesion"] for s in phantom.scatterers], dtype=bool)
26
27 # 1. Geometry
28 fig, ax = plt.subplots(figsize=(7, 6))
29 ax.scatter(x[~is_lesion] * 1e3, z[~is_lesion] * 1e3, s=3, alpha=0.35, label="background scatterers
↳ ")
30 ax.scatter(x[is_lesion] * 1e3, z[is_lesion] * 1e3, s=5, alpha=0.6, label="lesion-region scatterers
↳ ")
31 t = np.linspace(0, 2 * np.pi, 400)
32 ax.plot((lesion.center_x_m + lesion.radius_x_m * np.cos(t)) * 1e3,
33         (lesion.center_z_m + lesion.radius_z_m * np.sin(t)) * 1e3,
34         linewidth=2, label="lesion boundary")
35 ax.set(xlabel="Lateral x [mm]", ylabel="Depth z [mm]", title="2-D Acoustic Phantom Geometry")
36 ax.invert_yaxis(); ax.legend(); fig.tight_layout(); fig.savefig(OUT / "01_phantom_geometry.png", d
↳ pi=180); plt.close(fig)
37
38 # 2. Amplitude distribution
39 fig, ax = plt.subplots(figsize=(7, 4.5))
40 ax.hist(a[~is_lesion], bins=50, alpha=0.65, label="background")
41 if np.any(is_lesion):
42     ax.hist(a[is_lesion], bins=30, alpha=0.65, label="lesion region")
43 ax.set(xlabel="Signed scatterer amplitude", ylabel="Count", title="Scatterer Amplitude Distributio
↳ n")
44 ax.legend(); fig.tight_layout(); fig.savefig(OUT / "02_scatterer_amplitudes.png", dpi=180); plt.cl
↳ ose(fig)
45
46 # 3. Example waveform
47 fs = 40e6
48 n = 256
49 t_s = np.arange(n) / fs
50 f0 = 5e6
51 window = np.exp(-((t_s - 3.0 / f0) / (1.2 / f0)) ** 2)
52 incident = np.sin(2 * np.pi * f0 * t_s) * window
53 scatterer = phantom.scatterers[len(phantom.scatterers) // 2]
54 scattered = tissue_interaction(incident, scatterer)
55 fig, ax = plt.subplots(figsize=(8, 4.5))
56 ax.plot(t_s * 1e6, incident, label="incident")
57 ax.plot(t_s * 1e6, scattered, label="scattered")

```

```

58 ax.set(xlabel="Time [μs]", ylabel="Relative pressure amplitude", title="Example Incident / Scatter
    ↳ ed Waveform")
59 ax.legend(); fig.tight_layout(); fig.savefig(OUT / "03_waveform.png", dpi=180); plt.close(fig)
60
61 # 4. Background vs lesion-region scattering strength map (diagnostic only)
62 fig, ax = plt.subplots(figsize=(7, 6))
63 sc = ax.scatter(x * 1e3, z * 1e3, c=np.abs(a), s=4)
64 ax.plot((lesion.center_x_m + lesion.radius_x_m * np.cos(t)) * 1e3,
65         (lesion.center_z_m + lesion.radius_z_m * np.sin(t)) * 1e3,
66         linewidth=2)
67 fig.colorbar(sc, ax=ax, label="|scatterer amplitude|")
68 ax.set(xlabel="Lateral x [mm]", ylabel="Depth z [mm]", title="Acoustic Scatterer Strength (Not a B
    ↳ -mode Image)")
69 ax.invert_yaxis(); fig.tight_layout(); fig.savefig(OUT / "04_scattering_strength_map.png", dpi=180
    ↳ ); plt.close(fig)
70
71 print(f"Generated {phantom.num_scatterers} scatterers")
72 print(f"Lesion-region scatterers: {int(is_lesion.sum())}")
73 print(f"Plots written to: {OUT.resolve()}")

```

code/tests/conftest.py

```

1 import sys
2 from pathlib import Path
3 sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

```

code/tests/test_tissue_interaction.py

```

1 import numpy as np
2 import pytest
3
4 from tissue_interaction import interaction_gain, pressure_reflection_coefficient, tissue_interacti
    ↳ on
5
6
7 def test_equal_impedance_has_zero_interface_reflection():
8     assert pressure_reflection_coefficient(1.63, 1.63) == 0.0
9
10
11 def test_reflection_coefficient_matches_normal_incidence_formula():
12     z1, z2 = 1.5, 1.8
13     expected = (z2 - z1) / (z2 + z1)
14     assert np.isclose(pressure_reflection_coefficient(z1, z2), expected)
15
16
17 def test_existing_amp_only_phantom_remains_compatible():
18     wave = np.array([0.0, 1.0, -0.5, 0.25])
19     out = tissue_interaction(wave, {"x": 0.0, "z": 0.02, "amp": 0.4})
20     np.testing.assert_allclose(out, 0.4 * wave)
21
22
23 def test_output_shape_is_preserved():
24     wave = np.sin(np.linspace(0, 10, 100))
25     out = tissue_interaction(wave, {"amp": 0.7})
26     assert out.shape == wave.shape
27     assert np.all(np.isfinite(out))
28
29
30 def test_impedance_contrast_can_modify_scattering_gain():
31     g_equal = interaction_gain({"amp": 1.0, "background_impedance_mrayl": 1.63, "impedance_mrayl":
    ↳ 1.63})
32     g_contrast = interaction_gain({"amp": 1.0, "background_impedance_mrayl": 1.63, "impedance_mray
    ↳ l": 2.0})
33     assert g_equal == 1.0
34     assert g_contrast > g_equal
35
36
37 def test_empty_phantom_has_no_scatterers():
38     from phantom import PhantomConfig, generate_liver_phantom
39     p = generate_liver_phantom(PhantomConfig(scatterer_density_m2=0.0, seed=7))
40     assert p.num_scatterers == 0
41
42
43 def test_single_scatterer_has_predictable_echo():

```

```

44     wave = np.array([1.0, -2.0, 0.5])
45     out = tissue_interaction(wave, {"amp": 0.25})
46     np.testing.assert_allclose(out, 0.25 * wave)
47
48
49 def test_multiple_scatterers_can_be_composited_linearly():
50     wave = np.array([1.0, 2.0])
51     a = tissue_interaction(wave, {"amp": 0.2})
52     b = tissue_interaction(wave, {"amp": -0.1})
53     np.testing.assert_allclose(a + b, 0.1 * wave)
54
55
56 def test_same_seed_gives_identical_phantom():
57     from phantom import PhantomConfig, generate_liver_phantom
58     cfg = PhantomConfig(seed=123, scatterer_density_m2=10_000)
59     a = generate_liver_phantom(cfg)
60     b = generate_liver_phantom(cfg)
61     assert a.scatterers == b.scatterers
62
63
64 def test_different_seed_gives_different_phantom():
65     from phantom import PhantomConfig, generate_liver_phantom
66     a = generate_liver_phantom(PhantomConfig(seed=1, scatterer_density_m2=10_000))
67     b = generate_liver_phantom(PhantomConfig(seed=2, scatterer_density_m2=10_000))
68     assert a.scatterers != b.scatterers
69
70
71 def test_lesion_changes_local_scattering_statistics():
72     from phantom import LesionProperties, PhantomConfig, generate_liver_phantom
73     lesion = LesionProperties(center_z_m=0.035, radius_x_m=0.008, radius_z_m=0.008, echo_class="hy
    poechoic")
74     p = generate_liver_phantom(PhantomConfig(seed=99, scatterer_density_m2=80_000, lesion=lesion))
75     lesion_amps = np.array([float(s["amp"]) for s in p.scatterers if s["is_lesion"]])
76     background_amps = np.array([float(s["amp"]) for s in p.scatterers if not s["is_lesion"]])
77     assert lesion_amps.size > 0
78     assert background_amps.size > 0
79     assert np.std(lesion_amps) < np.std(background_amps)
80
81
82 def test_shape_units_and_finiteness():
83     from phantom import PhantomConfig, generate_liver_phantom
84     p = generate_liver_phantom(PhantomConfig(seed=4, scatterer_density_m2=20_000))
85     for s in p.scatterers:
86         assert p.x_min_m <= s["x"] <= p.x_max_m
87         assert p.z_min_m <= s["z"] <= p.z_max_m
88         assert np.isfinite(float(s["amp"]))
89
90
91 def test_invalid_waveform_is_rejected():
92     with pytest.raises(ValueError):
93         tissue_interaction(np.ones((2, 2)), {"amp": 1.0})

```

run_team4.py

```

1  """Team 4 (Tissue Interaction) - one-command demo runner.
2
3  Run from this folder:
4      python3 run_team4.py      (macOS / Linux)
5      python  run_team4.py      (Windows)
6
7  What it does (the team's original code in code/ is NOT changed, only called):
8      Step 1  runs the team's validate_phantom.py and collects its 4 plots
9      Step 2  runs the team's 13 unit tests (without needing pytest)
10     Step 3  demo: one 8 MHz incident pulse -> tissue_interaction() -> reflected pulse
11             for several scatterers, plus reflection coefficients for example impedances
12     Step 4  generates liver phantoms (background + 4 lesion types) and plots them
13     Step 5  small 1-D speckle illustration (many scatterer echoes added together)
14 Everything is written to the output/ folder; sample inputs are written to input/.
15 """
16 import csv
17 import importlib.util
18 import json
19 import os
20 import subprocess
21 import sys
22 import time

```

```

23 import traceback
24 import types
25 from contextlib import contextmanager
26 from dataclasses import asdict
27 from pathlib import Path
28
29 sys.dont_write_bytecode = True # keep code/ clean (no __pycache__)
30
31 import matplotlib
32
33 matplotlib.use("Agg")
34 import matplotlib.pyplot as plt # noqa: E402
35 import numpy as np # noqa: E402
36 from scipy.signal import hilbert # noqa: E402
37
38 HERE = Path(__file__).resolve().parent
39 CODE = HERE / "code"
40 INP = HERE / "input"
41 OUT = HERE / "output"
42 INP.mkdir(exist_ok=True)
43 OUT.mkdir(exist_ok=True)
44 sys.path.insert(0, str(CODE))
45
46 from phantom import LesionProperties, PhantomConfig, generate_liver_phantom # noqa: E402
47 from tissue_interaction import ( # noqa: E402
48     interaction_gain, pressure_reflection_coefficient, tissue_interaction,
49 )
50
51
52 # ----- console log
53 class Tee:
54     def __init__(self, *streams):
55         self.streams = streams
56
57     def write(self, s):
58         for st in self.streams:
59             st.write(s)
60
61     def flush(self):
62         for st in self.streams:
63             st.flush()
64
65
66 LOG_PATH = OUT / "console_log.txt"
67 _log = open(LOG_PATH, "w", encoding="utf-8")
68 _real_stdout = sys.stdout
69 sys.stdout = Tee(_real_stdout, _log)
70
71
72 def banner(text):
73     print()
74     print("=" * 78)
75     print(text)
76     print("=" * 78)
77
78
79 def save_console_png(text, out_png, title="Terminal - python3 run_team4.py", width_chars=100):
80     """Save the console text as a screenshot-style PNG (dark terminal look)."""
81     lines = []
82     for line in text.rstrip("\n").splitlines():
83         while len(line) > width_chars:
84             lines.append(line[:width_chars]); line = " -> " + line[width_chars:]
85         lines.append(line)
86     h = 0.62 + 0.165 * len(lines)
87     fig = plt.figure(figsize=(0.083 * width_chars + 0.4, h))
88     fig.patch.set_facecolor("#1e1e1e")
89     fig.text(0.012, 1 - 0.28 / h, "o o o " + title, color="#9a9a9a", fontsize=9, family="DejaVu
↳ Sans Mono", va="center")
90     for i, ln in enumerate(lines):
91         fig.text(0.012, 1 - (0.55 + 0.165 * i) / h, ln, color="#e6e6e6", fontsize=8.6,
92                 family="DejaVu Sans Mono", va="top")
93     fig.savefig(out_png, dpi=130, facecolor=fig.get_facecolor())
94     plt.close(fig)
95
96
97 t_start = time.time()
98 banner("NITK-UsoundSim | Team 4: Tissue Interaction | demo run")

```

```

99 print(f"Python {sys.version.split()[0]} numpy {np.__version__} matplotlib {matplotlib.__version__}")
100 print(f"Running in folder: {HERE}")
101 print("Team code: code/      Sample inputs: input/      Results: output/")
102
103 # ===== STEP 1
104 banner("STEP 1: run the team's validate_phantom.py (original script, unchanged)")
105 env = dict(os.environ, MPLBACKEND="Agg", PYTHONDONTWRITEBYTECODE="1")
106 # validate_phantom.py writes into ./validation_plots relative to the working
107 # folder, so we run it with output/ as working folder -> output/validation_plots/
108 proc = subprocess.run([sys.executable, str(CODE / "validate_phantom.py")], cwd=str(OUT), env=env,
109                       capture_output=True, text=True)
110 print("[validate_phantom.py says]")
111 for line in proc.stdout.strip().splitlines():
112     print("    " + line)
113 if proc.returncode != 0:
114     print("    ERROR (exit code %d):" % proc.returncode)
115     print(proc.stderr)
116 vp = sorted((OUT / "validation_plots").glob("*.png"))
117 print(f"Plots made by validate_phantom.py ({len(vp)}):")
118 for p in vp:
119     print(f"    output/validation_plots/{p.name}")
120
121 # ===== STEP 2
122 banner("STEP 2: run the team's unit tests (code/tests/test_tissue_interaction.py)")
123 try:
124     import pytest # noqa: F401
125     print("pytest is installed; still running the test functions directly for a clear report.")
126 except ImportError:
127     # The test file does 'import pytest' and uses only pytest.raises(...).
128     # pytest is not required for this demo, so we provide a tiny stand-in
129     # that does the same job as pytest.raises.
130     print("pytest is not installed -> using a tiny built-in stand-in for 'pytest.raises'.")
131     stub = types.ModuleType("pytest")
132
133     @contextmanager
134     def _raises(exc_type):
135         try:
136             yield
137         except exc_type:
138             return
139         raise AssertionError(f"DID NOT RAISE {exc_type.__name__}")
140
141     stub.raises = _raises
142     sys.modules["pytest"] = stub
143
144 def load_module(name, path):
145     spec = importlib.util.spec_from_file_location(name, path)
146     mod = importlib.util.module_from_spec(spec)
147     spec.loader.exec_module(mod)
148     return mod
149
150
151 load_module("conftest", CODE / "tests" / "conftest.py") # only adds code/ to sys.path
152 tests = load_module("test_tissue_interaction", CODE / "tests" / "test_tissue_interaction.py")
153 test_names = [n for n in dir(tests) if n.startswith("test_") and callable(getattr(tests, n))]
154 test_names.sort(key=lambda n: getattr(tests, n).__code__.co_firstlineno) # file order
155 passed = 0
156 for i, name in enumerate(test_names, 1):
157     try:
158         getattr(tests, name)()
159         passed += 1
160         print(f"    [{i:2d}] PASS {name}")
161     except Exception: # noqa: BLE001
162         print(f"    [{i:2d}] FAIL {name}")
163         print("    " + traceback.format_exc().strip().splitlines()[-1])
164 print(f"Result: {passed}/{len(test_names)} tests passed")
165
166 # ===== STEP 3
167 banner("STEP 3: demo - one 8 MHz incident pulse through tissue_interaction()")
168 fs = 100e6 # sampling rate [Hz] (chosen for this demo)
169 f0 = 8e6 # centre frequency [Hz]
170 n = 200 # samples -> 2 microseconds
171 t = np.arange(n) / fs
172 t0 = 1.0e-6 # pulse centre [s]
173 sigma = 0.15e-6 # Gaussian envelope width [s]

```

```

175 incident = np.exp(-((t - t0) ** 2) / (2 * sigma ** 2)) * np.sin(2 * np.pi * f0 * (t - t0))
176 with open(INP / "incident_pulse_8MHz.csv", "w", newline="") as fh:
177     w = csv.writer(fh)
178     w.writerow(["time_us", "pressure_relative"])
179     for ti, pi in zip(t, incident):
180         w.writerow([f"{ti * 1e6:.4f}", f"{pi:.6f}"])
181 print(f"Incident pulse: Gaussian-modulated sine, f0 = {f0/1e6:.0f} MHz, fs = {fs/1e6:.0f} MHz, "
182       f"{n} samples ({n/fs*1e6:.1f} us), envelope sigma = {sigma*1e6:.2f} us")
183 print(f" peak |p_incident| = {np.max(np.abs(incident)):.4f} saved to input/incident_pulse_8MHz.
      ↪ csv")
184
185 ZBG = 1.63 # background (liver-like) impedance used by the team's code [MRayl]
186 cases = [
187     ("A amp = 1.0 (plain scatterer)", {"amp": 1.0}),
188     ("B amp = 0.5 (weaker scatterer)", {"amp": 0.5}),
189     ("C amp = -0.8 (negative amp -> flipped)", {"amp": -0.8}),
190     ("D amp = 1.0, Z = 1.38 (fat-like)", {"amp": 1.0, "background_impedance_mrayl": ZBG, "impedan
      ↪ ce_mrayl": 1.38}),
191     ("E amp = 1.0, Z = 7.8 (bone-like)", {"amp": 1.0, "background_impedance_mrayl": ZBG, "impedan
      ↪ ce_mrayl": 7.8}),
192     ("F amp = 1.0, scale = 0.35 (hypoechoic)", {"amp": 1.0, "interaction_scale": 0.35}),
193     ("G amp = 1.0, scale = 1.8 (hyperechoic)", {"amp": 1.0, "interaction_scale": 1.8}),
194     ("H amp = 1.0, scale = 0.0 (anechoic)", {"amp": 1.0, "interaction_scale": 0.0}),
195 ]
196 with open(INP / "demo_scatterers.json", "w") as fh:
197     json.dump({"note": "Scatterer dictionaries passed to tissue_interaction() in Step 3. "
198           "Impedances are example textbook-style values chosen for the demo.",
199           "cases": [{"label": lab, "scatterer": sc} for lab, sc in cases]}, fh, indent=2)
200 print("Scatterer cases (saved to input/demo_scatterers.json):")
201 print(f" {'case':44s} {'gain G':>8s} {'peak |p_out|':>13s} {'same shape?':>12s}")
202 reflected = []
203 for lab, sc in cases:
204     out = tissue_interaction(incident, sc)
205     g = interaction_gain(sc)
206     reflected.append(out)
207     print(f" {lab:44s} {g:8.4f} {np.max(np.abs(out)):13.4f} {str(out.shape == incident.shape):>12
      ↪ s}")
208 print(" (G = amp * (1 + |R|) * interaction_scale; output = G * incident, no time shift)")
209
210 # Figure: incident vs reflected for each case
211 fig, axes = plt.subplots(4, 2, figsize=(11, 10), sharex=True, sharey=True)
212 for ax, (lab, sc), out in zip(axes.ravel(), cases, reflected):
213     ax.plot(t * 1e6, incident, color="0.6", lw=1.2, label="incident")
214     ax.plot(t * 1e6, out, color="C3", lw=1.4, label="reflected")
215     ax.set_title(f"{lab}\nG = {interaction_gain(sc):.3f}", fontsize=9)
216     ax.grid(alpha=0.3)
217 for ax in axes[-1]:
218     ax.set_xlabel("Time [μs]")
219 for ax in axes[:, 0]:
220     ax.set_ylabel("Relative pressure")
221 axes[0, 0].legend(fontsize=8, loc="upper right")
222 axes[0, 0].set_xlim(0.3, 1.7)
223 fig.suptitle("Team 4 demo: 8 MHz incident pulse vs reflected pulse from tissue_interaction()", fon
      ↪ tsize=11)
224 fig.tight_layout()
225 fig.savefig(OUT / "demo_01_incident_vs_reflected.png", dpi=150)
226 plt.close(fig)
227
228 fig, ax = plt.subplots(figsize=(8, 3.2))
229 ax.plot(t * 1e6, incident, color="C0")
230 ax.set_xlabel="Time [μs]", ylabel="Relative pressure", title="Sample input: Gaussian-modulated 8 M
      ↪ Hz incident pulse")
231 ax.grid(alpha=0.3); fig.tight_layout(); fig.savefig(OUT / "demo_00_incident_pulse.png", dpi=150);
      ↪ plt.close(fig)
232
233 print()
234 print("Pressure reflection coefficient R = (Z2 - Z1)/(Z2 + Z1), Z1 = soft tissue 1.63 MRayl")
235 print("(the Z2 values below are example textbook-style values chosen as inputs for this demo)")
236 pairs = [("fat", 1.38), ("water", 1.48), ("blood", 1.61), ("soft tissue (same)", 1.63),
237          ("muscle", 1.70), ("bone", 7.80), ("air", 0.0004)]
238 print(f" {'Z2 material':20s} {'Z2 [MRayl]':>11s} {'R':>9s} {'R^2 (energy)':>13s} {'1+|R|':>7s} s
      ↪ ign")
239 r_rows = []
240 for name, z2 in pairs:
241     r = pressure_reflection_coefficient(1.63, z2)
242     r_rows.append((name, z2, r))
243     sign = "inverted (-)" if r < 0 else ("none" if r == 0 else "same (+)")

```

```

244     print(f"  {name:20s} {z2:11.4f} {r:9.4f} {r*r:13.4f} {1+abs(r):7.4f} {sign}")
245
246 fig, ax = plt.subplots(figsize=(8, 3.6))
247 names = [f"{nm}\n{z:g}" for nm, z, _ in r_rows]
248 vals = [r for _, r in r_rows]
249 ax.bar(names, vals, color=["C3" if v < 0 else "C0" for v in vals])
250 ax.axhline(0, color="k", lw=0.8)
251 for i, v in enumerate(vals):
252     ax.text(i, v + (0.03 if v >= 0 else -0.03), f"{v:.3f}", ha="center", va="bottom" if v >= 0 else
        ↪ e "top", fontsize=8)
253 ax.set(ylabel="R (pressure)", title="Reflection coefficient from soft tissue (1.63 MRayl) into mat
        ↪ erial Z2 [MRayl]",
254        ylim=(-1.15, 1.0))
255 ax.tick_params(axis="x", labelsize=8)
256 fig.tight_layout(); fig.savefig(OUT / "demo_02_reflection_coefficients.png", dpi=150); plt.close(f
        ↪ ig)
257
258 # ===== STEP 4
259 banner("STEP 4: liver phantoms with generate_liver_phantom() - background + 4 lesion types")
260 geo = dict(center_x_m=0.0, center_z_m=0.035, radius_x_m=0.007, radius_z_m=0.008)
261 phantom_cases = [
262     ("background (no lesion)", None),
263     ("hypoechoic lesion", LesionProperties(echo_class="hypoechoic", **geo)),
264     ("isoechoic lesion", LesionProperties(echo_class="isoechoic", **geo)),
265     ("hyperechoic lesion", LesionProperties(echo_class="hyperechoic", **geo)),
266     ("anechoic cyst (scale 0.0)", LesionProperties(echo_class="hypoechoic", scattering_scale=0.0,
        ↪ **geo)),
267 ]
268 DENSITY, SEED = 120_000.0, 2026
269 cfg_dump = []
270 phantoms = []
271 for lab, les in phantom_cases:
272     cfg = PhantomConfig(seed=SEED, scatterer_density_m2=DENSITY, lesion=les)
273     ph = generate_liver_phantom(cfg)
274     phantoms.append((lab, cfg, ph))
275     cfg_dump.append({"name": lab, "config": asdict(cfg)})
276 with open(INP / "phantom_configs.json", "w") as fh:
277     json.dump({"note": "PhantomConfig values passed to generate_liver_phantom() in Step 4 (SI unit
        ↪ s, metres).",
278               "phantoms": cfg_dump}, fh, indent=2)
279 print("Phantom configs saved to input/phantom_configs.json")
280 c0 = phantoms[0][1]
281 area = (c0.x_max_m - c0.x_min_m) * (c0.z_max_m - c0.z_min_m)
282 print(f"Region x = {c0.x_min_m*1e3:.0f}..{c0.x_max_m*1e3:.0f} mm, z = {c0.z_min_m*1e3:.0f}..{c0.z_
        ↪ max_m*1e3:.0f} mm, "
283       f"area = {area*1e4:.1f} cm^2, density = {DENSITY:,.0f} per m^2, seed = {SEED}")
284 print(f"Expected N = round(density * area) = {int(round(DENSITY*area))}")
285 print("Lesion: ellipse centre (0, 35) mm, radii 7 x 8 mm")
286 print(f" {'phantom':28s} {'N':>5s} {'in lesion':>9s} {'scale':>6s} {'std(bg)':>8s} {'std(lesion)'
        ↪ ':>11s} {'ratio':>6s}")
287 scale_of = {"hypoechoic": 0.35, "isoechoic": 1.0, "hyperechoic": 1.8}
288 for lab, cfg, ph in phantoms:
289     a = np.array([s["amp"] for s in ph.scatterers])
290     les = np.array([s["is_lesion"] for s in ph.scatterers])
291     if cfg.lesion is None:
292         print(f"  {lab:28s} {ph.num_scatterers:5d} {'-':>9s} {'-':>6s} {np.std(a):8.3f} {'-':>11s}
        ↪ {'-':>6s}")
293     else:
294         sc = cfg.lesion.scattering_scale if cfg.lesion.scattering_scale is not None else scale_of[
        ↪ cfg.lesion.echo_class]
295         sb, sl = np.std(a[~les]), np.std(a[les])
296         print(f"  {lab:28s} {ph.num_scatterers:5d} {int(les.sum()):9d} {sc:6.2f} {sb:8.3f} {sl:11.
        ↪ 3f} {sl/sb:6.2f}")
297         with open(OUT / f"phantom_{lab.split()[0]}_scatterers.csv", "w", newline="") as fh:
298             wr = csv.writer(fh)
299             wr.writerow(["x_mm", "z_mm", "amp", "is_lesion"])
300             for s in ph.scatterers:
301                 wr.writerow([f"{s['x']*1e3:.4f}", f"{s['z']*1e3:.4f}", f"{s['amp']:.5f}", int(s["is_le
        ↪ sion"])]])
302 print("Same seed -> the SAME positions in all 5 phantoms; only amplitudes inside the lesion change
        ↪ .")
303 print("Scatterer lists saved as output/phantom_<type>_scatterers.csv")
304
305 tt = np.linspace(0, 2 * np.pi, 300)
306 lim = max(np.max(np.abs([s["amp"] for s in ph.scatterers])) for _, _, ph in phantoms)
307 fig, axes = plt.subplots(1, 5, figsize=(17, 4.2), sharey=True, layout="constrained")
308 for ax, (lab, cfg, ph) in zip(axes, phantoms):

```

```

309     x = np.array([s["x"] for s in ph.scatterers]) * 1e3
310     z = np.array([s["z"] for s in ph.scatterers]) * 1e3
311     a = np.array([s["amp"] for s in ph.scatterers])
312     sc_ = ax.scatter(x, z, c=a, cmap="coolwarm", vmin=-lim, vmax=lim, s=10 + 25 * np.abs(a), edgec
↳ olors="k",
313                     linewidths=0.2)
314     if cfg.lesion is not None:
315         L = cfg.lesion
316         ax.plot((L.center_x_m + L.radius_x_m * np.cos(tt)) * 1e3, (L.center_z_m + L.radius_z_m * n
↳ p.sin(tt)) * 1e3,
317               "k--", lw=1.2)
318     ax.set_title(f"{lab}\nN = {ph.num_scatterers}", fontsize=9)
319     ax.set_xlabel("Lateral x [mm]"); ax.set_aspect("equal")
320     ax.set_xlim(-25, 25); ax.set_ylim(60, 10)
321 axes[0].set_ylabel("Depth z [mm]")
322 fig.colorbar(sc_, ax=axes, label="signed scatterer amplitude", shrink=0.9, pad=0.01)
323 fig.suptitle("generate_liver_phantom(): scatterer positions coloured by amplitude (dot size ~ |amp
↳ |)", fontsize=11)
324 fig.savefig(OUT / "demo_03_phantoms_scatterers.png", dpi=150, bbox_inches="tight")
325 plt.close(fig)
326
327 fig, axes = plt.subplots(1, 5, figsize=(17, 3.6), sharey=True)
328 bins = np.linspace(-lim, lim, 25)
329 for ax, (lab, cfg, ph) in zip(axes, phantoms):
330     a = np.array([s["amp"] for s in ph.scatterers])
331     les = np.array([s["is_lesion"] for s in ph.scatterers])
332     ax.hist(a[~les], bins=bins, color="C0", alpha=0.7, label="background")
333     if les.any():
334         ax.hist(a[les], bins=bins, color="C3", alpha=0.8, label="inside lesion")
335     ax.set_title(lab, fontsize=9); ax.set_xlabel("amplitude"); ax.legend(fontsize=7)
336 axes[0].set_ylabel("count")
337 fig.suptitle("Scatterer amplitude histograms (Gaussian, mean 0) - background vs lesion region", fo
↳ ntsize=11)
338 fig.tight_layout(); fig.savefig(OUT / "demo_04_amplitude_histograms.png", dpi=150); plt.close(fig)
339
340 # ===== STEP 5
341 banner("STEP 5: 1-D speckle illustration (many tiny scatterers -> interference)")
342 print("Note: the time delays below are added by this demo only; in the full simulator")
343 print("delays are Team 3's job (propagation). Team 4's function only scales each echo.")
344 rng = np.random.default_rng(7)
345 c = 1540.0
346 n_line = 1200
347 t_line = np.arange(n_line) / fs # 12 us of RF line
348 pulse_short = incident[int((t0 - 4 * sigma) * fs): int((t0 + 4 * sigma) * fs)]
349 rf = np.zeros(n_line + len(pulse_short))
350 n_sc = 80
351 depths = rng.uniform(1.5e-3, 7.5e-3, n_sc) # scatterers between 1.5 and 7.5 mm
352 amps = rng.normal(0.0, 1.0, n_sc)
353 for d, a_ in zip(depths, amps):
354     echo = tissue_interaction(pulse_short, {"amp": a_})
355     k = int(round(2 * d / c * fs))
356     rf[k:k + len(echo)] += echo
357 rf = rf[:n_line]
358 env_ = np.abs(hilbert(rf))
359 lam = c / f0
360 print(f"{n_sc} scatterers placed at random depths 1.5-7.5 mm, Gaussian amplitudes, wavelength = {l
↳ am*1e3:.3f} mm")
361 print(f"About {n_sc / (6.0e-3 / lam):.1f} scatterers per wavelength -> echoes overlap and interfer
↳ e")
362 print(f"Envelope (2-10 us): mean = {env_[200:1000].mean():.3f}, std/mean = {env_[200:1000].std()/e
↳ nv_[200:1000].mean():.3f}"
363       " (random bright/dark pattern = speckle)")
364 fig, axes = plt.subplots(2, 1, figsize=(9, 5.2), sharex=True)
365 axes[0].stem(2 * depths / c * 1e6, amps, basefmt=" ", markerfmt=".")
366 axes[0].set_ylabel="scatterer amp", title="Random scatterers (position -> round-trip time 2z/c)")
367 axes[1].plot(t_line * 1e6, rf, color="0.55", lw=0.8, label="summed RF echoes")
368 axes[1].plot(t_line * 1e6, env_, color="C3", lw=1.4, label="envelope (|Hilbert|)")
369 axes[1].set_xlabel="Time [μs]", ylabel="pressure", title="Sum of all echoes: interference gives sp
↳ eckle")
370 axes[1].legend(fontsize=8)
371 for ax in axes:
372     ax.grid(alpha=0.3)
373 fig.tight_layout(); fig.savefig(OUT / "demo_05_speckle_1d.png", dpi=150); plt.close(fig)
374
375 # ===== SUMMARY
376 banner("SUMMARY")
377 print(f"validate_phantom.py : {'OK' if proc.returncode == 0 else 'FAILED'} ({len(vp)} plots)")

```

```
378 print(f"Unit tests          : {passed}/{len(test_names)} passed")
379 print("Output files:")
380 for p in sorted(OUT.rglob("*")):
381     if p.is_file() and p.name not in ("console_log.txt", "console_screenshot.png"):
382         print(f"    output/{p.relative_to(OUT).as_posix()}")
383 print("    output/console_log.txt")
384 print("    output/console_screenshot.png")
385 print(f"Total runtime: {time.time() - t_start:.1f} s")
386
387 sys.stdout = _real_stdout
388 _log.close()
389 save_console_png(LOG_PATH.read_text(encoding="utf-8"), OUT / "console_screenshot.png")
390 print("Console log and screenshot saved in output/.")
```

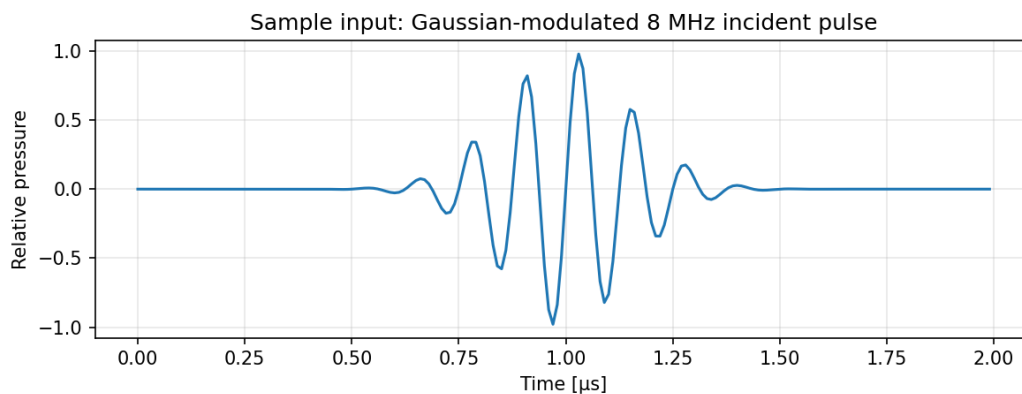
4 Input

The sample inputs used by the run (all in input/)

4.1 Incident pulse: input/incident_pulse_8MHz.csv

A Gaussian-modulated sine made by the runner: $f_0 = 8$ MHz, sampling rate 100 MHz, 200 samples ($2 \mu\text{s}$), centre at $1 \mu\text{s}$, envelope $\sigma = 0.15 \mu\text{s}$, peak $|p| = 0.9783$. Columns: time_us, pressure_relative.

$$p(t) = e^{-(t-t_0)^2/(2\sigma^2)} \sin(2\pi f_0(t-t_0))$$



The sample incident pulse (input to Step 3).

4.2 Scatterer cases: input/demo_scatterers.json

Case	Scatterer dictionary passed to tissue_interaction()
A	{amp: 1.0}
B	{amp: 0.5}
C	{amp: -0.8}
D	{amp: 1.0, background_impedance_mrayl: 1.63, impedance_mrayl: 1.38} (fat-like)
E	{amp: 1.0, background_impedance_mrayl: 1.63, impedance_mrayl: 7.8} (bone-like)
F	{amp: 1.0, interaction_scale: 0.35}
G	{amp: 1.0, interaction_scale: 1.8}
H	{amp: 1.0, interaction_scale: 0.0}

Note on impedance values

The impedances used for fat (1.38), water (1.48), blood (1.61), muscle (1.70), bone (7.8) and air (0.0004 MRayl) are example textbook-style values chosen as inputs for this demo. They are not part of the team's code; only the 1.63 MRayl background comes from the team's code.

4.3 Phantom configurations: input/phantom_configs.json

Setting	Value (same for all 5 phantoms)
Region	$x = -25$ to 25 mm, $z = 10$ to 60 mm
Density / seed	120,000 per m^2 / 2026

Setting	Value (same for all 5 phantoms)
Background impedance / amp std	1.63 MRayl / 1.0
Lesion (not in background phantom)	ellipse centre (0, 35) mm, radii 7 mm × 8 mm
Lesion types	hypoechoic, isoechoic, hyperechoic, and anechoic (hypoechoic with scattering_scale = 0.0)

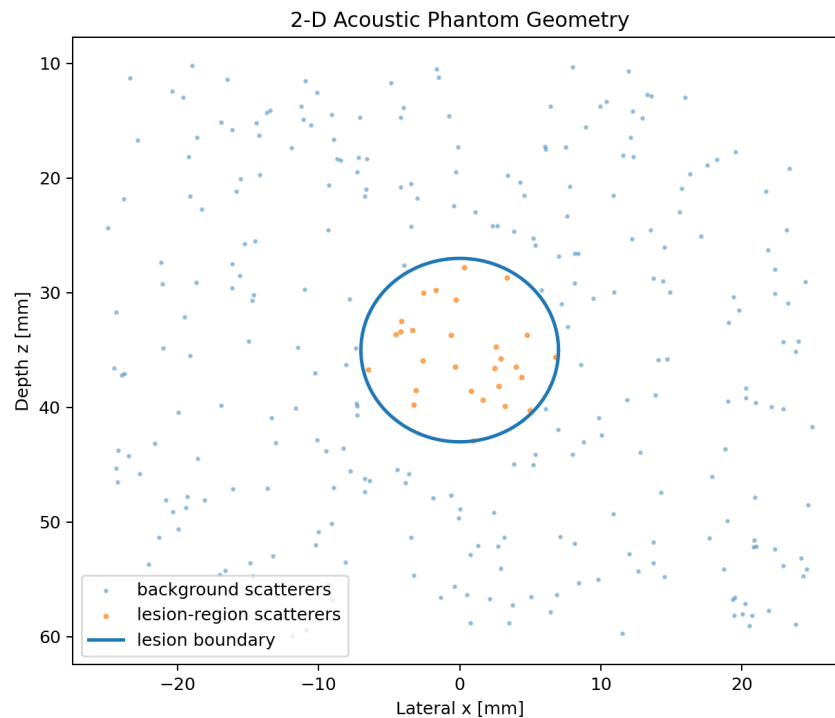
validate_phantom.py has its own input values written inside the script (seed 2026, density 120,000, hypoechoic lesion at (0, 35) mm with radii 7 × 8 mm, 5 MHz test pulse).

5 Output

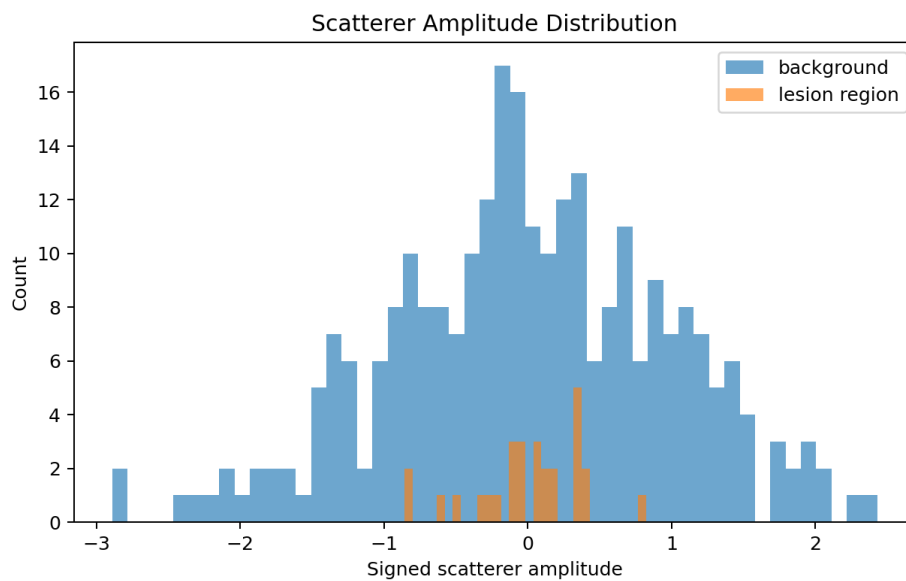
Every result produced by run_team4.py

5.1 Team's validation plots (output/validation_plots/)

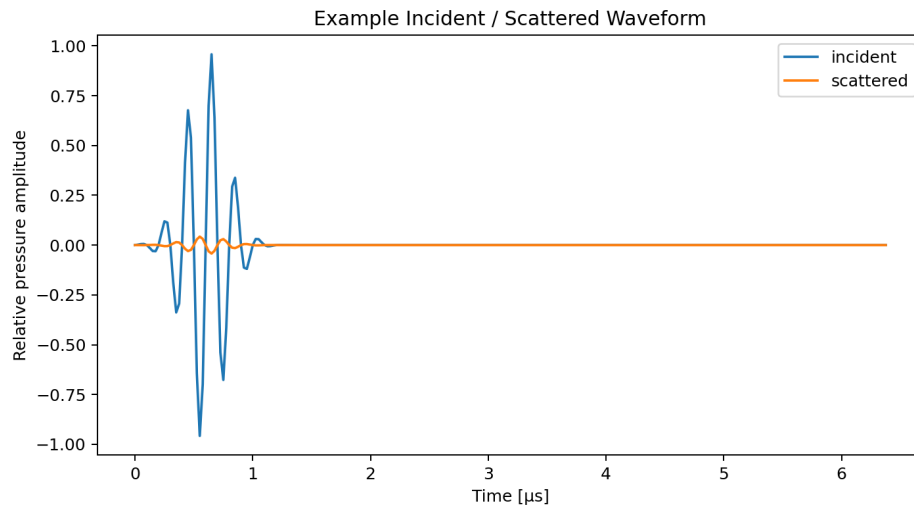
validate_phantom.py printed: Generated 300 scatterers, Lesion-region scatterers: 28.



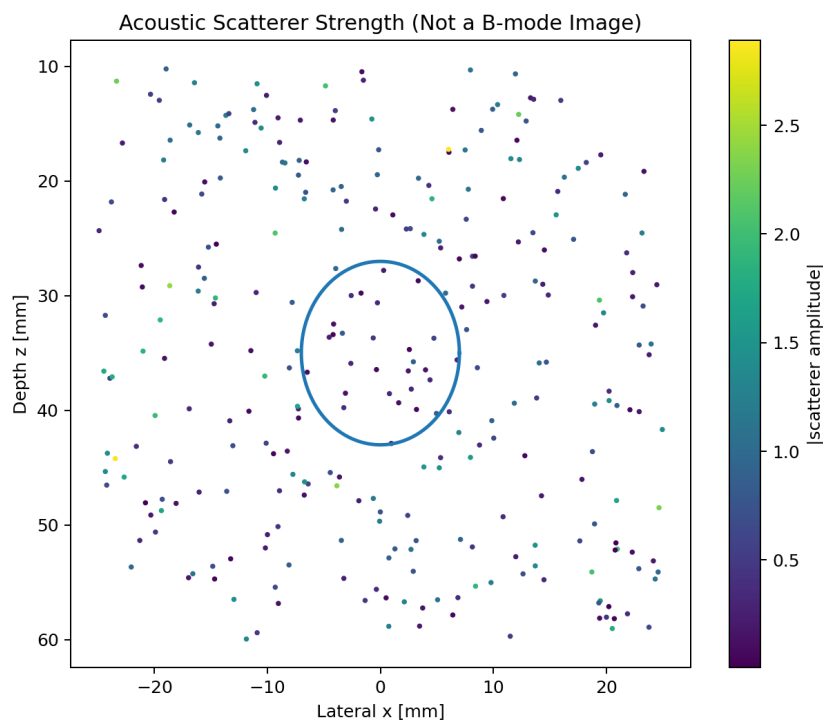
01_phantom_geometry.png: the 300 scatterer positions; the 28 inside the hypoechoic lesion are shown in a second colour, with the lesion boundary. Depth increases downwards like on a scanner.



02_scatterer_amplitudes.png: the background amplitudes form a bell (Gaussian) shape around 0; the lesion amplitudes are squeezed closer to 0 (scale 0.35).

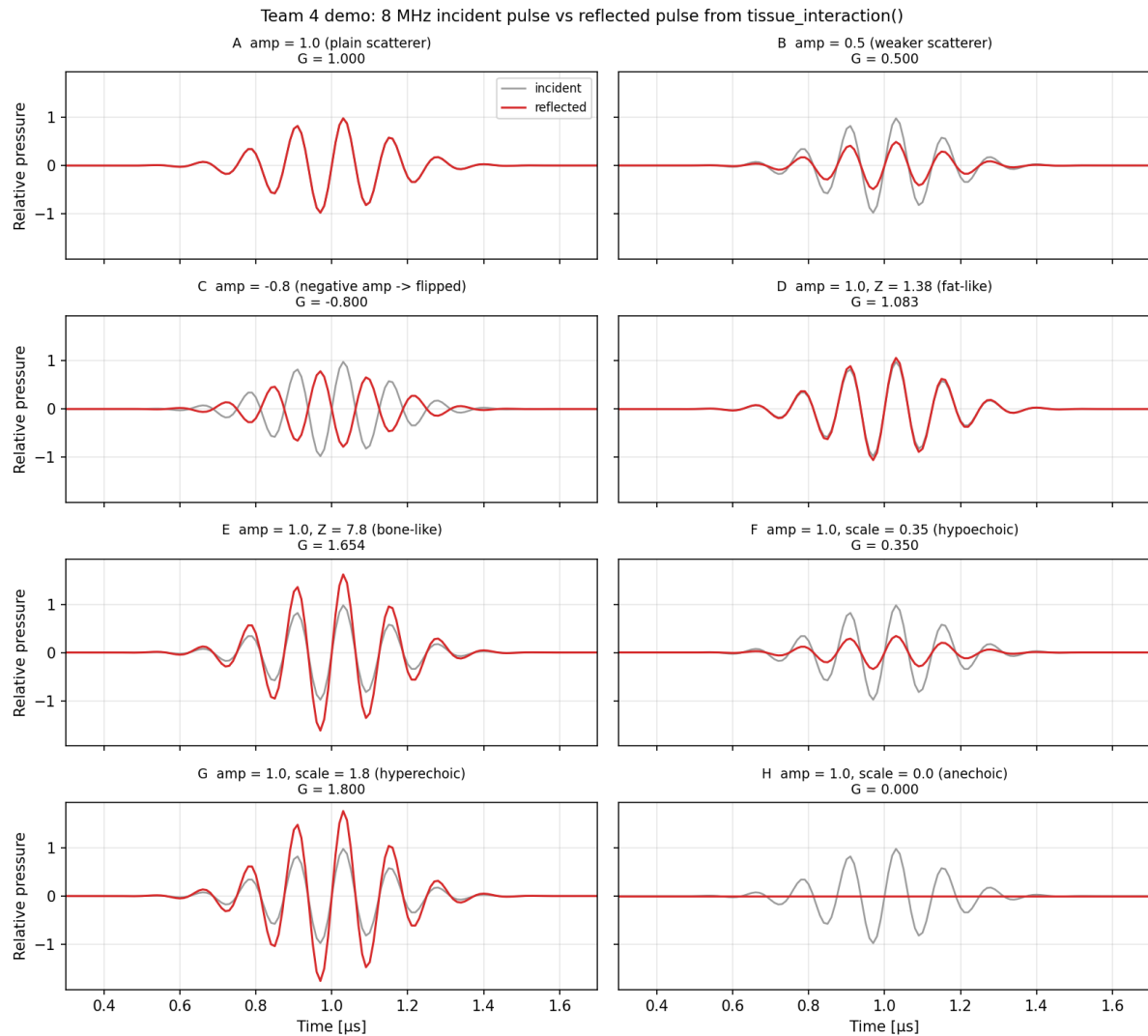


03_waveform.png: 5 MHz incident pulse and the scattered pulse for the middle scatterer in the list. That scatterer has $\text{amp} = -0.04448$ (from `output/phantom_hypoechoic_scatterers.csv`, which uses the same settings), so the echo is small and flipped.



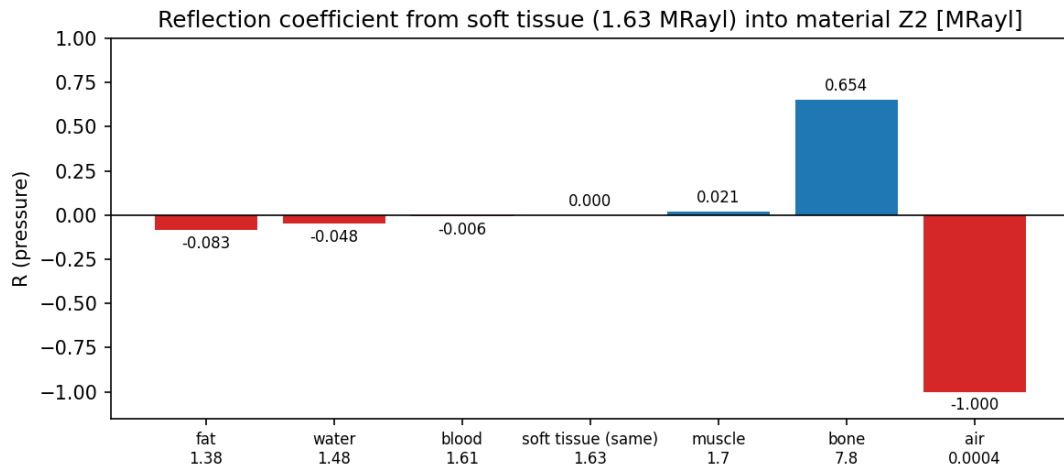
04_scattering_strength_map.png: colour shows $|\text{amp}|$. Inside the lesion the points are darker (weaker). This is an acoustic map, not a B-mode image.

5.2 Pulse demo (Step 3)



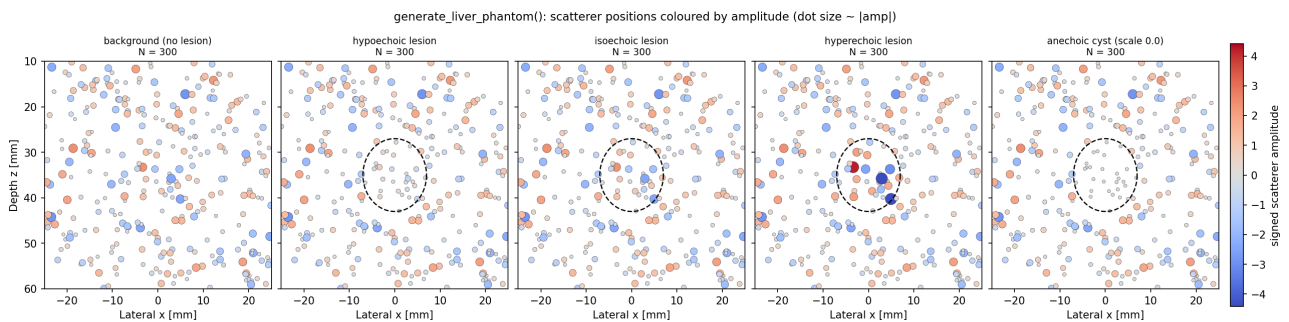
demo_01: grey = incident, red = reflected. A: identical. B: half size. C: flipped and 0.8x. D: fat-like impedance gives $G = 1.0831$. E: bone-like gives $G = 1.6543$. F/G: scale 0.35 and 1.8. H: scale 0 gives no echo. The pulse shape and timing never change: the model is a pure gain.

Case	Gain G	Peak $ p_{out} $
A amp 1.0	1.0000	0.9783
B amp 0.5	0.5000	0.4891
C amp -0.8	-0.8000	0.7826
D $Z = 1.38$	1.0831	1.0595
E $Z = 7.8$	1.6543	1.6183
F scale 0.35	0.3500	0.3424
G scale 1.8	1.8000	1.7609
H scale 0.0	0.0000	0.0000

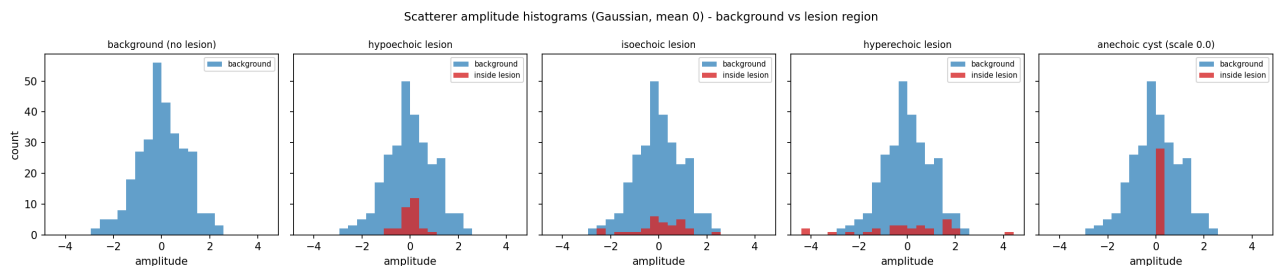


demo_02: R from soft tissue (1.63 MRayl) into other example materials. Red bars are negative (phase inversion). Fat -0.0831, blood -0.0062, bone +0.6543, air -0.9995.

5.3 Phantoms (Step 4)



demo_03: the five phantoms. The same seed gives the same 300 positions every time; only the 28 amplitudes inside the dashed lesion change: smaller (hypo), unchanged in spread (iso), larger (hyper), zero (anechoic).

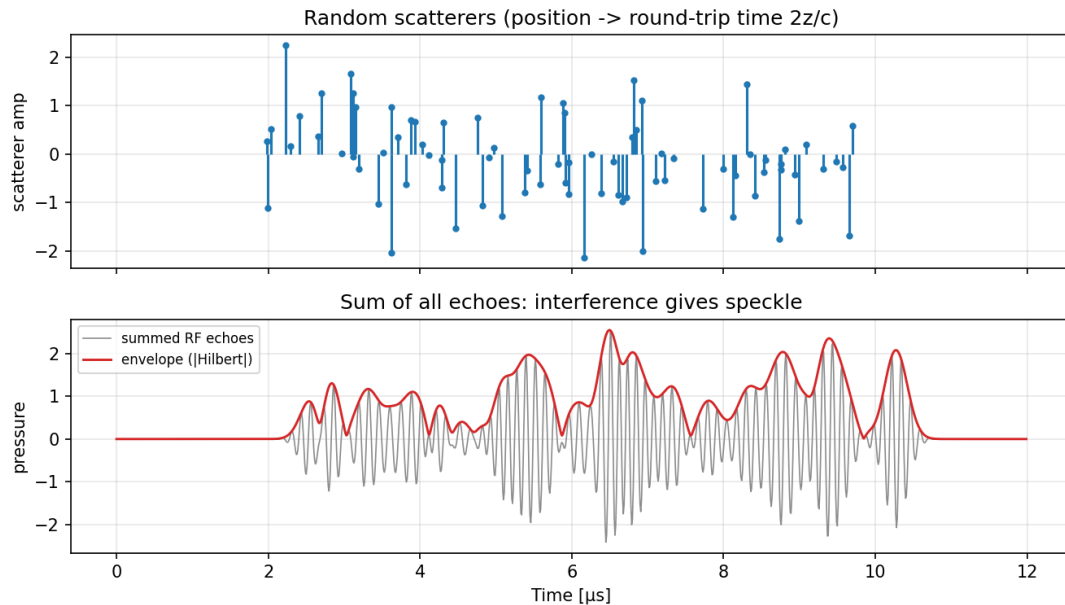


demo_04: amplitude histograms. Blue = background, red = inside lesion. The anechoic lesion is a single bar at 0.

Phantom	N	In lesion	Scale	std(bg)	std(lesion)	Ratio
background	300	-	-	0.986 (all)	-	-
hypoechoic	300	28	0.35	0.976	0.377	0.39
isoechoic	300	28	1.00	0.976	1.076	1.10
hyperechoic	300	28	1.80	0.976	1.937	1.98
anechoic	300	28	0.00	0.976	0.000	0.00

The ratio $\text{std(lesion)}/\text{std(bg)}$ is close to the scale factor but not equal, because only 28 random samples are inside the lesion (e.g. isoechoic gives 1.10 instead of 1.00).

5.4 Speckle illustration (Step 5)



demo_05: top: 80 random scatterers (about 2.6 per wavelength of 0.192 mm). Bottom: their echoes, each made by `tissue_interaction()`, added together. The envelope goes up and down randomly ($\text{std}/\text{mean} = 0.614$): speckle.

5.5 Other output files

- `phantom_<type>_scatterers.csv`: `x_mm`, `z_mm`, `amp`, `is_lesion` for each of the 5 phantoms.
- `console_log.txt` and `console_screenshot.png`: the console output below.

5.6 Console output (real, captured)

python3 run_team4.py

```
=====
NITK-UsoundSim | Team 4: Tissue Interaction | demo run
=====
Python 3.12.4 numpy 1.26.4 matplotlib 3.8.3
Running in folder: /Users/darshil/Downloads/NITK-UsoundSim_Team_Projects/Team_4_Tissue_Interaction
Team code: code/ Sample inputs: input/ Results: output/

=====
STEP 1: run the team's validate_phantom.py (original script, unchanged)
=====
[validate_phantom.py says]
  Generated 300 scatterers
  Lesion-region scatterers: 28
  Plots written to: /Users/darshil/Downloads/NITK-UsoundSim_Team_Projects/Team_4_Tissue_Interaction/output/validation_plots
  ↳ put/validation_plots
Plots made by validate_phantom.py (4):
  output/validation_plots/01_phantom_geometry.png
  output/validation_plots/02_scatterer_amplitudes.png
  output/validation_plots/03_waveform.png
  output/validation_plots/04_scattering_strength_map.png

=====
STEP 2: run the team's unit tests (code/tests/test_tissue_interaction.py)
=====
pytest is not installed -> using a tiny built-in stand-in for 'pytest.raises'.
[ 1] PASS test_equal_impedance_has_zero_interface_reflection
[ 2] PASS test_reflection_coefficient_matches_normal_incidence_formula
[ 3] PASS test_existing_amp_only_phantom_remains_compatible
[ 4] PASS test_output_shape_is_preserved
[ 5] PASS test_impedance_contrast_can_modify_scattering_gain
[ 6] PASS test_empty_phantom_has_no_scatterers
[ 7] PASS test_single_scatterer_has_predictable_echo
```

```
[ 8] PASS  test_multiple_scatterers_can_be_composited_linearly
[ 9] PASS  test_same_seed_gives_identical_phantom
[10] PASS  test_different_seed_gives_different_phantom
[11] PASS  test_lesion_changes_local_scattering_statistics
[12] PASS  test_shape_units_and_finiteness
[13] PASS  test_invalid_waveform_is_rejected
Result: 13/13 tests passed

=====
STEP 3: demo - one 8 MHz incident pulse through tissue_interaction()
=====
Incident pulse: Gaussian-modulated sine, f0 = 8 MHz, fs = 100 MHz, 200 samples (2.0 us), envelope sigma
↳ = 0.15 us
peak |p_incident| = 0.9783  saved to input/incident_pulse_8MHz.csv
Scatterer cases (saved to input/demo_scatterers.json):
case      gain G    peak |p_out|  same shape?
A amp = 1.0 (plain scatterer)      1.0000      0.9783      True
B amp = 0.5 (weaker scatterer)     0.5000      0.4891      True
C amp = -0.8 (negative amp -> flipped) -0.8000      0.7826      True
D amp = 1.0, Z = 1.38 (fat-like)    1.0831      1.0595      True
E amp = 1.0, Z = 7.8 (bone-like)    1.6543      1.6183      True
F amp = 1.0, scale = 0.35 (hypoechoic) 0.3500      0.3424      True
G amp = 1.0, scale = 1.8 (hyperechoic) 1.8000      1.7609      True
H amp = 1.0, scale = 0.0 (anechoic)   0.0000      0.0000      True
(G = amp * (1 + |R|) * interaction_scale; output = G * incident, no time shift)

Pressure reflection coefficient  $R = (Z_2 - Z_1)/(Z_2 + Z_1)$ ,  $Z_1$  = soft tissue 1.63 MRayl
(the  $Z_2$  values below are example textbook-style values chosen as inputs for this demo)
Z2 material      Z2 [MRayl]      R    R^2 (energy)  1+|R|  sign
fat              1.3800      -0.0831      0.0069  1.0831  inverted (-)
water            1.4800      -0.0482      0.0023  1.0482  inverted (-)
blood            1.6100      -0.0062      0.0000  1.0062  inverted (-)
soft tissue (same) 1.6300      0.0000      0.0000  1.0000  none
muscle           1.7000      0.0210      0.0004  1.0210  same (+)
bone             7.8000      0.6543      0.4281  1.6543  same (+)
air              0.0004      -0.9995      0.9990  1.9995  inverted (-)

=====
STEP 4: liver phantoms with generate_liver_phantom() - background + 4 lesion types
=====
Phantom configs saved to input/phantom_configs.json
Region x = -25..25 mm, z = 10..60 mm, area = 25.0 cm^2, density = 120,000 per m^2, seed = 2026
Expected N = round(density * area) = 300
Lesion: ellipse centre (0, 35) mm, radii 7 x 8 mm
phantom      N in lesion  scale  std(bg)  std(lesion)  ratio
background (no lesion)  300      -      0.986      -      -
hypoechoic lesion      300      28  0.35  0.976      0.377  0.39
isoechoic lesion       300      28  1.00  0.976      1.076  1.10
hyperechoic lesion     300      28  1.80  0.976      1.937  1.98
anechoic cyst (scale 0.0) 300      28  0.00  0.976      0.000  0.00
Same seed -> the SAME positions in all 5 phantoms; only amplitudes inside the lesion change.
Scatterer lists saved as output/phantom_<type>_scatterers.csv

=====
STEP 5: 1-D speckle illustration (many tiny scatterers -> interference)
=====
Note: the time delays below are added by this demo only; in the full simulator
delays are Team 3's job (propagation). Team 4's function only scales each echo.
80 scatterers placed at random depths 1.5-7.5 mm, Gaussian amplitudes, wavelength = 0.192 mm
About 2.6 scatterers per wavelength -> echoes overlap and interfere
Envelope (2-10 us): mean = 1.000, std/mean = 0.614 (random bright/dark pattern = speckle)

=====
SUMMARY
=====
validate_phantom.py : OK (4 plots)
Unit tests          : 13/13 passed
Output files:
  output/demo_00_incident_pulse.png
  output/demo_01_incident_vs_reflected.png
  output/demo_02_reflection_coefficients.png
  output/demo_03_phantoms_scatterers.png
  output/demo_04_amplitude_histograms.png
  output/demo_05_speckle_1d.png
```

```
output/phantom_anechoic_scatterers.csv
output/phantom_background_scatterers.csv
output/phantom_hyperechoic_scatterers.csv
output/phantom_hypoechoic_scatterers.csv
output/phantom_isoechoic_scatterers.csv
output/validation_plots/01_phantom_geometry.png
output/validation_plots/02_scatterer_amplitudes.png
output/validation_plots/03_waveform.png
output/validation_plots/04_scattering_strength_map.png
output/console_log.txt
output/console_screenshot.png
Total runtime: 2.6 s
```

6 How to run

Step by step, on any computer

1. Install Python 3.9 or newer from python.org (tested with Python 3.12.4). On Windows tick "Add Python to PATH" during install.
2. Open a terminal: Terminal on macOS/Linux, Command Prompt or PowerShell on Windows.
3. Go into the folder: `cd path/to/Team_4_Tissue_Interaction`
4. Install the packages once: `python3 -m pip install -r requirements.txt` (Windows: `python -m pip install -r requirements.txt`)
5. Run: `python3 run_team4.py` (Windows: `python run_team4.py`)
6. Open the output/ folder for the plots, CSV files, console log and console screenshot. Sample inputs are in input/.

Expected runtime: a few seconds (measured 2.6 s inside the script, about 7 s including Python start-up). pytest is not needed for run_team4.py.

Optional: run the team's files yourself

- Tests with real pytest (pytest is in requirements.txt): `cd code && python3 -m pytest tests` runs the same 13 tests.
- Validation script alone: `cd code && python3 validate_phantom.py` (writes validation_plots/ inside code/).
- VS Code: open the team folder (File > Open Folder), choose a Python interpreter that has the packages, open a file and press Run. `.vscode/settings.json` sets the import path to code/.

Troubleshooting

Problem	What to do
python3: command not found (Windows)	Use python instead of python3
No module named numpy / scipy / matplotlib	Run the pip install step again
pip says "externally managed environment"	<code>python3 -m venv venv</code> , activate it, then pip install again
No files in output/	Run the command from inside the folder; check the folder is writable
A test shows FAIL	Make sure the files in code/ were not edited

7 Check yourself

Viva-style questions with short answers

What is acoustic impedance and what is the value used for liver here?

$Z = \rho c$ (density $\times$ speed of sound). The code uses 1.63 MRayl as the background soft-tissue value.

Why does the echo from fat come back flipped, but the echo from bone does not?

$R = (Z_2 - Z_1)/(Z_2 + Z_1)$. Fat (1.38) has lower Z than tissue (1.63), so $R = -0.0831 < 0$: phase inversion. Bone (7.8) is higher, so $R = +0.6543 > 0$: same sign.

What does `tissue_interaction()` return, and what does it NOT do?

It returns $G \times$ incident wave, same shape. It adds no time delay, no attenuation and no beamforming; those are done by other teams.

Why are scatterer positions uniform and amplitudes Gaussian with mean 0?

Tissue has no preferred location for tiny structures (uniform), and many small random differences in both directions add up to a zero-mean Gaussian. This gives realistic speckle.

How is an anechoic cyst made with this code, and what does it look like?

Use a lesion with `scattering_scale = 0.0`. All amplitudes inside become 0, so there are no echoes: the region is black on the image.

What is speckle?

The random bright/dark pattern made by interference of echoes from many scatterers smaller than the wavelength inside one resolution cell. It is a property of coherent imaging, not electronic noise.